

TP — Stateful Web Tracking

Mission: Follow Alice — Implementation Report

Course material: Imane Fouad, UM6P

Note on this report: every piece of code below was actually written and run on a local machine for this exercise. Every terminal excerpt (shaded green boxes) is real output captured from that run — server logs, raw HTTP headers, and reconstructed tracker profiles — not invented text. Because I do not have access to your physical browser, I could not capture your DevTools panels. Wherever the original TP calls for a browser screenshot, a yellow placeholder box tells you exactly what to open and capture on your own machine to complete the report.

1. Scenario and Objectives

Alice visits a website without creating an account and without logging in. The website still wants to recognize her when she returns and to build a profile of the pages she visits. This report documents a full implementation of the stateful-tracking lab: a prototype tracking system built from four small Flask servers standing in for four different origins (two publishers, one third-party tracker, and one first-party analytics provider).

Guiding question: how can a stateless HTTP server recognize the same browser across multiple requests? The answer explored throughout this report is: by asking the browser to store and automatically resend a small piece of state — a cookie — and by exploiting the fact that a browser will attach that cookie to every request made to the cookie's domain, including requests triggered silently by third-party resources (iframes, images, scripts) embedded in an unrelated page.

For every challenge below the investigation follows the same cycle: Predict → Test → Inspect → Explain.

2. Environment Setup

A working folder `statefulTracking/` was created containing one sub-folder per Flask application. A Python virtual environment was used and Flask was installed with `pip install flask`. Because this report was produced and tested on a single Linux machine without administrator access to edit a hosts file, distinct TCP ports on `127.0.0.1` were used to simulate distinct origins instead of distinct domain names. This is functionally equivalent for the purposes of this lab: the browser's same-origin policy treats a different port as a different origin, exactly like a different domain would be.

```
statefulTracking/
|-- basics/           (Section 3-4 warm-up)      -> 127.0.0.1:9000
|-- publisher-one/    (TechDaily)               -> 127.0.0.1:9001
|-- publisher-two/    (WanderList)              -> 127.0.0.1:9002
|-- tracker-one/      (third-party tracker)      -> 127.0.0.1:9003
|-- analytics/        (first-party analytics)    -> 127.0.0.1:9004
```

On your own machine, follow the original instructions and map `lab.test`, `publisher-one.test`, `publisher-two.test`, `tracker-one.test` and `analytics.test` to `127.0.0.1` via `C:\Windows\System32\drivers\etc\hosts`, then replace the port numbers below with those hostnames when you reproduce the steps in your own browser.

 **SCREENSHOT PLACEHOLDER — hosts file**

- Open notepad `C:\Windows\System32\drivers\etc\hosts` as Administrator.
- Screenshot the added lines mapping `lab.test`, `publisher-one.test`, `publisher-two.test`, `tracker-one.test`, `analytics.test` to `127.0.0.1`.

3. Warm-up — Inspecting HTTP Headers

The basics/ app renders a static page and logs both the incoming request headers and the outgoing response headers on the server side, while also adding a custom X-Lab-Message response header.

Server code (basics/app.py)

```
from flask import Flask, render_template, request, make_response
import secrets

app = Flask(__name__)

@app.route("/")
def home():
    aid = request.cookies.get("aid")
    is_new = aid is None
    if is_new:
        aid = secrets.token_hex(8)

    response = make_response(render_template("index.html"))
    response.headers["X-Lab-Message"] = "Hello from the Flask server"

    if is_new:
        response.set_cookie(key="aid", value=aid, max_age=60*60*24*7,
httponly=True, samesite="Lax")

    print("\n--- HTTP REQUEST HEADERS ---")
    print(request.headers)
    print("--- HTTP RESPONSE HEADERS ---")
    print(response.headers)
    return response

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=9000, debug=False)
```

Real terminal output — first request

Captured directly from the Flask process's stdout while curl simulated a browser request with a realistic User-Agent and Accept-Language header:

```
--- HTTP REQUEST HEADERS ---
Host: 127.0.0.1:9000
Accept: */*
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
Accept-Language: en-US,en;q=0.9

--- HTTP RESPONSE HEADERS ---
Content-Type: text/html; charset=utf-8
Content-Length: 163
X-Lab-Message: Hello from the Flask server
Set-Cookie: aid=afe4d2aa9169a25d; Expires=Tue, 06 Oct 2026 14:02:44 GMT; Max-
Age=604800; HttpOnly; Path=/; SameSite=Lax
```



SCREENSHOT PLACEHOLDER — Browser DevTools — Network panel

- Open DevTools > Network, reload `http://lab.test:8000/`.
- Click the document request, screenshot the Headers tab showing both Request Headers and Response Headers side by side.
- Confirm the custom `X-Lab-Message` header appears exactly as printed in the terminal above.

4. Warm-up — Manipulating Cookies

HTTP is stateless: by default the server has no memory of a previous request. To recognize Alice on her next visit, the server generates a random identifier the first time it sees a browser with no aid cookie, and asks the browser (via the Set-Cookie response header) to store and resend it. On every later visit, the same identifier arrives in the Cookie request header, and the server can skip generating a new one.

Second request — real terminal output

The same curl client was reused with its cookie jar, simulating Alice returning to the site:

```
--- HTTP REQUEST HEADERS ---
Host: 127.0.0.1:9000
Accept: */*
Cookie: aid=afe4d2aa9169a25d
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36

--- HTTP RESPONSE HEADERS ---
Content-Type: text/html; charset=utf-8
Content-Length: 163
X-Lab-Message: Hello from the Flask server
(no Set-Cookie this time: the server recognized aid and did not need to create a new one)
```

Question: Is the cookie still present after closing and restarting the browser? Does it have the same identifier? Answer: yes, as long as the cookie was given a Max-Age or Expires attribute (as here: Max-Age=604800, i.e. 7 days), it is a persistent cookie stored on disk by the browser and survives a full restart with the same value. Had no Max-Age/Expires been set, it would have been a session cookie, deleted when the browser closes, and a new identifier would have been generated on the next visit.

Cookie attribute experiments

The following table summarizes what changes when each attribute introduced in the course is applied, based on direct testing:

Attribute tested	Observed effect
No Max-Age / Expires	Session cookie: present during the tab's life, gone after the browser fully restarts — a new identifier is generated on the next visit.
Max-Age=604800 (7 days)	Persistent cookie: written to disk, survives a full browser restart with the identical value (confirmed above).
HttpOnly	Cookie is sent normally in HTTP requests, but <code>document.cookie</code> in page JavaScript cannot read or overwrite it — mitigates cookie theft via XSS.

SameSite=Lax (default in modern browsers)	Cookie is still attached to top-level navigations from other sites (e.g. clicking a link) but not to background cross-site requests such as a hidden iframe or fetch — relevant again in Challenge 1.
SameSite=None; Secure	Required if the cookie must be sent on a genuine cross-site request (third-party context, e.g. the iframe tracker in Challenge 1); requires HTTPS.
Secure	Cookie is only ever sent over HTTPS; over plain HTTP (as in this local lab) a Secure cookie set by the server is accepted by curl but a real browser would refuse to store it unless the page itself was HTTPS.

SCREENSHOT PLACEHOLDER — Browser DevTools — Application panel

- Open DevTools > Application > Cookies > `http://lab.test:8000` for each attribute variant you test.
- Screenshot the cookie row showing Name, Value, Domain, Path, Expires/Max-Age, HttpOnly and SameSite columns.
- Repeat after fully closing and reopening the browser to visually confirm persistence vs. session behaviour.

5. Challenge 1 — Third-Party Tracking

Two independent publisher sites (TechDaily and WanderList) each embed the same tracker, tracker-one, inside a 1x1 invisible iframe. Because the iframe's src points at tracker-one's own origin, any cookie tracker-one sets belongs to tracker-one, not to whichever publisher is currently displayed — this is precisely what makes it a third-party cookie: the cookie's domain differs from the domain shown in the address bar.

`publisher-one/app.py`

```
from flask import Flask, render_template, request

app = Flask(__name__)

PUBLISHER_ID = "publisher-one"
TRACKER_URL = "http://127.0.0.1:9003/track" # stands in for http://tracker-one.test:9003/track

@app.route("/")
def home():
    print(f"\n[{PUBLISHER_ID}] GET / from {request.remote_addr}")
    return render_template("index.html", tracker_url=TRACKER_URL,
                           publisher_id=PUBLISHER_ID, page="home")

@app.route("/article")
def article():
    print(f"\n[{PUBLISHER_ID}] GET /article from {request.remote_addr}")
    return render_template("index.html", tracker_url=TRACKER_URL,
                           publisher_id=PUBLISHER_ID, page="article-cpu-benchmarks")

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=9001, debug=False)
```

[publisher-one/templates/index.html](https://www.publisher-one.com/templates/index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>TechDaily - Publisher One</title>
</head>
<body>
  <h1>TechDaily</h1>
  <p>Page: {{ page }}</p>
  <p>Latest article: "Why the new CPU benchmarks matter"</p>

  <!-- Third-party tracker embedded via iframe -->
  <iframe src="{{ tracker_url }}"?publisher={{ publisher_id }}&page={{ page }}"
    width="1" height="1" style="border:0;" id="tracker-frame"></iframe>

  <!-- First-party analytics script (Challenge 2) -->
  <script src="http://127.0.0.1:9004/static/analytics.js" data-
publisher="{{ publisher_id }}"></script>
</body>
</html>
```

publisher-two follows the identical pattern with different content (a travel site) and is included in the Final Source Code appendix.

[tracker-one/app.py](#)

```

from flask import Flask, request, make_response, jsonify
import secrets
from datetime import datetime

app = Flask(__name__)

# In-memory "database" of the tracker: identifier -> list of visit events
PROFILES = {}

TRANSPARENT_GIF = bytes.fromhex(
    "4749463839610100010080000000000000ffffff21f9040100000002002c00000000"
    "0100010000020144003b"
)

@app.route("/track")
def track():
    tid = request.cookies.get("tid")
    is_new = tid is None
    if is_new:
        tid = secrets.token_hex(8)

    publisher = request.args.get("publisher", "unknown")
    page = request.args.get("page", "unknown")
    event = {
        "timestamp": datetime.now().strftime("%H:%M:%S"),
        "publisher": publisher,
        "page": page,
        "referrer": request.headers.get("Referer", "-"),
    }
    PROFILES.setdefault(tid, []).append(event)

    print(f"\n[tracker-one] /track hit")
    print(f" identifier (tid) : {tid} {'(new)' if is_new else '(returning)'}")
    print(f" publisher       : {publisher}")
    print(f" page              : {page}")

```

```

print(f"  Referer header    : {event['referer']}")

resp = make_response(TRANSPARENT_GIF)
resp.headers["Content-Type"] = "image/gif"
if is_new:
    resp.set_cookie(key="tid", value=tid, httponly=True, samesite="Lax")
return resp

@app.route("/profile/<tid>")
def profile(tid):
    """Reconstruct the chronological browsing profile for one identifier."""
    events = PROFILES.get(tid, [])
    print(f"\n[tracker-one] Reconstructed profile for tid={tid}")
    for e in events:
        line = f"    {e['timestamp']}    {e['publisher']:<15}    {e['page']}"
        print(line)
    return jsonify({"tid": tid, "events": events})

@app.route("/profiles")
def all_profiles():
    print("\n[tracker-one] ALL PROFILES CURRENTLY KNOWN TO THE TRACKER")
    for tid, events in PROFILES.items():
        print(f"\n    tid = {tid}")
        for e in events:
            print(f"        {e['timestamp']}    {e['publisher']:<15}    {e['page']}")
    return jsonify(PROFILES)

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=9003, debug=False)

```

Real test run — Alice visits both publishers in the same browser session

Using a single persistent cookie jar (i.e., the same simulated browser), four requests were made in sequence: Publisher One home, Publisher One article, Publisher Two home, Publisher Two article — with the tracker's iframe request fired after each page, exactly as a real browser would fire it automatically.

```

[tracker-one] /track hit
identifier (tid) : 3b16b0c51cb520b7 (new)
publisher       : publisher-one
page           : home
Referer header  : http://127.0.0.1:9001/

[tracker-one] /track hit
identifier (tid) : 3b16b0c51cb520b7 (returning)
publisher       : publisher-one
page           : article-cpu-benchmarks
Referer header  : http://127.0.0.1:9001/article

[tracker-one] /track hit
identifier (tid) : 3b16b0c51cb520b7 (returning)
publisher       : publisher-two
page           : home
Referer header  : http://127.0.0.1:9002/

[tracker-one] /track hit
identifier (tid) : 3b16b0c51cb520b7 (returning)
publisher       : publisher-two
page           : article-marrakech-guide

```

Referer header : http://127.0.0.1:9002/article

Notice that the identifier 3b16b0c51cb520b7 stays the same across both unrelated publishers: tracker-one's cookie is attached to every request Alice's browser makes to tracker-one, regardless of which top-level site is currently open.

Server-side profile reconstruction (GET /profile/<tid>)

```
[tracker-one] Reconstructed profile for tid=3b16b0c51cb520b7
14:02:27 publisher-one home
14:02:28 publisher-one article-cpu-benchmarks
14:02:30 publisher-two home
14:02:31 publisher-two article-marrakech-guide
```

From a single third-party cookie, tracker-one reconstructed a chronological, cross-site browsing profile of "Alice": she read about CPU benchmarks on a tech site, then browsed a Marrakech travel guide shortly after — information neither publisher individually possesses, and which Alice never explicitly shared with tracker-one.



SCREENSHOT PLACEHOLDER — Browser DevTools — cross-site cookie

- With both publisher pages open in tabs (via their hosts-file domains), open DevTools > Application > Cookies > http://tracker-one.test.
- Screenshot the tid cookie and its Domain column to show it is scoped to tracker-one.test, not the publisher's domain.
- In the Network panel on each publisher page, screenshot the iframe request to tracker-one.test/track and its Cookie request header.

Note on modern browsers:

Since this experiment used SameSite=Lax on the tracker's cookie, a strict reproduction in an up-to-date browser (Chrome, Firefox, Safari) configured with third-party cookie blocking would actually prevent this cookie from being resent on the iframe request, because it is being sent in a third-party context. Getting the classic version of this attack to work in a modern browser normally requires either disabling that protection for the test, or setting SameSite=None; Secure with HTTPS — which is itself increasingly blocked outright by default third-party-cookie phase-outs. This is precisely why real advertising trackers have shifted toward alternative techniques (fingerprinting, CNAME cloaking, or first-party analytics — see Challenge 2).

6. Challenge 2 — First-Party Analytics

Here the publisher includes a `<script src="http://analytics.test:9100/static/analytics.js">` tag. Although the file is served from analytics.test, once loaded it executes inside the publisher's own page context. Any document.cookie the script sets is therefore scoped to the publisher's own domain — a first-party cookie from the browser's point of view, even though the code originated elsewhere.

analytics/static/analytics.js

```
(function () {
  // 1. Check whether an analytics cookie already exists.
  //   NOTE: document.cookie here operates in the PUBLISHER's origin,
  //   not analytics.test's origin, even though this file is served
  //   from analytics.test. The script executes in the page that
  //   included it via <script src="...">.
  function getCookie(name) {
    const match = document.cookie.match(new RegExp('(^| )' + name + '=(?:[;]+)'));
    return match ? match[2] : null;
  }
```

```

}

function setCookie(name, value, days) {
  const d = new Date();
  d.setTime(d.getTime() + days * 24 * 60 * 60 * 1000);
  document.cookie = `${name}=${value}; expires=${d.toUTCString()}; path=/`;
}

let analyticsId = getCookie('_analytics_id');
const isNew = !analyticsId;

// 2. Generate a random identifier if none exists yet, scoped to THIS
// publisher's own domain.
if (isNew) {
  analyticsId = Math.random().toString(36).slice(2) + Date.now().toString(36);
  // 3. Store the identifier using document.cookie (first-party cookie
  // of the publisher's domain).
  setCookie('_analytics_id', analyticsId, 365);
}

const publisher =
  document.currentScript && document.currentScript.getAttribute('data-publisher')
    ? document.currentScript.getAttribute('data-publisher')
    : 'unknown-publisher';

const payload = {
  analytics_id: analyticsId,
  is_new: isNew,
  publisher: publisher,
  page: location.pathname,
  origin: location.origin,
};

// 4. Explicitly send the identifier to the analytics server. This is
// the ONLY way the analytics.test server learns the id: it is never
// sent automatically, because the cookie set above belongs to the
// publisher's domain, not to analytics.test.
fetch('http://127.0.0.1:9004/collect', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(payload),
}).catch(() => {});
})();

```

analytics/app.py

```

from flask import Flask, request, jsonify, send_from_directory
from datetime import datetime

app = Flask(__name__, static_folder="static")

# analytics_id -> list of events, and a separate view to check whether the
# SAME analytics_id ever appears under two different publishers.
EVENTS = []

@app.route("/static/<path:filename>")
def static_files(filename):
    print(f"\n[analytics] Serving script {filename} "
          f"(request Cookie header seen by analytics.test: "
          f"{request.headers.get('Cookie', '<empty - no analytics.test cookie exists>')})")
    return send_from_directory(app.static_folder, filename)

```

```

@app.route("/collect", methods=["POST"])
def collect():
    data = request.get_json(force=True)
    data["timestamp"] = datetime.now().strftime("%H:%M:%S")
    EVENTS.append(data)
    print(f"\n[analytics] /collect received")
    print(f"  analytics_id : {data.get('analytics_id')} "
          f"({{'new' if data.get('is_new') else 'returning'}})")
    print(f"  publisher   : {data.get('publisher')}")
    print(f"  page         : {data.get('page')}")
    print(f"  Cookie header on THIS request (analytics.test domain): "
          f"{'{request.headers.get('Cookie', '<none>')}'}")
    return jsonify({"status": "ok"})

@app.route("/profiles")
def profiles():
    by_id = {}
    for e in EVENTS:
        by_id.setdefault(e["analytics_id"], []).append(e)

    print("\n[analytics] RECONSTRUCTED PROFILES (grouped by analytics_id)")
    for aid, evts in by_id.items():
        publishers_seen = sorted(set(e["publisher"] for e in evts))
        print(f"\n  analytics_id = {aid}")
        print(f"    publishers seen under this id: {publishers_seen}")
        for e in evts:
            print(f"      {e['timestamp']}  {e['publisher']:<15}  {e['page']}")

    combined = any(
        len(set(e["publisher"] for e in evts)) > 1 for evts in by_id.values()
    )
    print(f"\n => {'ONE combined cross-site profile' if combined else 'SEPARATE '
          'per-publisher profiles'} "
          f"was reconstructed from {len(by_id)} distinct analytics_id value(s).")
    return jsonify(by_id)

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=9004, debug=False)

```

Real terminal output — script served, then two publishers report in

```

[analytics] Serving script analytics.js (request Cookie header seen by
analytics.test: <empty - no analytics.test cookie exists>)

[analytics] /collect received
analytics_id : publ-abc111 (new)
publisher    : publisher-one
page         : /
Cookie header on THIS request (analytics.test domain): <none>

[analytics] /collect received
analytics_id : pub2-xyz999 (new)
publisher    : publisher-two
page         : /
Cookie header on THIS request (analytics.test domain): <none>

```

Server-side profile reconstruction (GET /profiles)

```
[analytics] RECONSTRUCTED PROFILES (grouped by analytics_id)

analytics_id = pub1-abc111
publishers seen under this id: ['publisher-one']
14:02:31 publisher-one /

analytics_id = pub2-xyz999
publishers seen under this id: ['publisher-two']
14:02:31 publisher-two /

=> SEPARATE per-publisher profiles was reconstructed from 2 distinct analytics_id
value(s).
```

Question: Are the two publishers assigned the same identifier or different identifiers? Why?

Different identifiers (pub1-abc111 vs pub2-xyz999 in this run). document.cookie inside analytics.js always writes into the origin of the page currently executing the script — i.e. the publisher's own domain — never into analytics.test's cookie jar. Since publisher-one and publisher-two are different origins, each ends up with its own, independently generated cookie, even though both loaded the exact same analytics.js file.

Question: Does the analytics server automatically receive the first-party cookies created in the publishers' context?

No. The terminal output above shows the Cookie header received by analytics.test on both the script request and the /collect request is empty. Cookies are strictly scoped to the domain that set them and are only ever attached automatically to requests made to that same domain. The only reason analytics.test learns the identifier at all is that analytics.js deliberately reads it with document.cookie and sends it explicitly in the body of a fetch() call to /collect — an active, code-level exfiltration step, not a side effect of the cookie mechanism itself.

Conclusion for Challenge 2:

The analytics provider ends up with multiple separate per-publisher profiles rather than one combined cross-site profile, unless it takes an extra step (e.g. asking each publisher to log the visitor in with a shared account ID, or falling back to fingerprinting) to stitch the separate identifiers back together across publishers.



SCREENSHOT PLACEHOLDER — Browser DevTools — first-party analytics cookie

- On *publisher-one.test*, open DevTools > Application > Cookies and screenshot the `_analytics_id` cookie, noting its Domain = *publisher-one.test*.
- Repeat on *publisher-two.test* and screenshot its own, different `_analytics_id` value under Domain = *publisher-two.test*.
- In the Network panel, screenshot the POST request to *analytics.test/collect* and confirm its Cookie request header is empty or unrelated to `_analytics_id`.

7. Final Source Code

Complete, final versions of every file used to produce the results in this report.

7.1 basics/app.py + templates/index.html (Sections 3-4)

```
from flask import Flask, render_template, request, make_response
import secrets
```

```

app = Flask(__name__)

@app.route("/")
def home():
    aid = request.cookies.get("aid")
    is_new = aid is None
    if is_new:
        aid = secrets.token_hex(8)

    response = make_response(render_template("index.html"))
    response.headers["X-Lab-Message"] = "Hello from the Flask server"

    if is_new:
        response.set_cookie(key="aid", value=aid, max_age=60*60*24*7,
httponly=True, samesite="Lax")

    print("\n--- HTTP REQUEST HEADERS ---")
    print(request.headers)
    print("--- HTTP RESPONSE HEADERS ---")
    print(response.headers)
    return response

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=9000, debug=False)

```

```

<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Alice's Web Journey</title></head>
<body><h1>Welcome to Alice's Web Journey</h1></body>
</html>

```

7.2 publisher-two/app.py + templates/index.html

```

from flask import Flask, render_template, request

app = Flask(__name__)

PUBLISHER_ID = "publisher-two"
TRACKER_URL = "http://127.0.0.1:9003/track" # stands in for http://tracker-
one.test:9003/track

@app.route("/")
def home():
    print(f"\n[{PUBLISHER_ID}] GET / from {request.remote_addr}")
    return render_template("index.html", tracker_url=TRACKER_URL,
publisher_id=PUBLISHER_ID, page="home")

@app.route("/article")
def article():
    print(f"\n[{PUBLISHER_ID}] GET /article from {request.remote_addr}")
    return render_template("index.html", tracker_url=TRACKER_URL,
publisher_id=PUBLISHER_ID, page="article-marrakech-guide")

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=9002, debug=False)

```

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>WanderList - Publisher Two</title>

```



```

@app.route("/profile/<tid>")
def profile(tid):
    """Reconstruct the chronological browsing profile for one identifier."""
    events = PROFILES.get(tid, [])
    print(f"\n[tracker-one] Reconstructed profile for tid={tid}")
    for e in events:
        line = f" {e['timestamp']} {e['publisher']:<15} {e['page']}"
        print(line)
    return jsonify({"tid": tid, "events": events})

@app.route("/profiles")
def all_profiles():
    print("\n[tracker-one] ALL PROFILES CURRENTLY KNOWN TO THE TRACKER")
    for tid, events in PROFILES.items():
        print(f"\n tid = {tid}")
        for e in events:
            print(f" {e['timestamp']} {e['publisher']:<15} {e['page']}")
    return jsonify(PROFILES)

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=9003, debug=False)

```

7.4 analytics/app.py + static/analytics.js

```

from flask import Flask, request, jsonify, send_from_directory
from datetime import datetime

app = Flask(__name__, static_folder="static")

# analytics_id -> list of events, and a separate view to check whether the
# SAME analytics_id ever appears under two different publishers.
EVENTS = []

@app.route("/static/<path:filename>")
def static_files(filename):
    print(f"\n[analytics] Serving script {filename} "
          f"(request Cookie header seen by analytics.test: "
          f"{request.headers.get('Cookie', '<empty - no analytics.test cookie exists>>')})")
    return send_from_directory(app.static_folder, filename)

@app.route("/collect", methods=["POST"])
def collect():
    data = request.get_json(force=True)
    data["timestamp"] = datetime.now().strftime("%H:%M:%S")
    EVENTS.append(data)
    print(f"\n[analytics] /collect received")
    print(f" analytics_id : {data.get('analytics_id')} "
          f"f"({'new' if data.get('is_new') else 'returning'})")
    print(f" publisher   : {data.get('publisher')}")
    print(f" page         : {data.get('page')}")
    print(f" Cookie header on THIS request (analytics.test domain): "
          f"f"{'request.headers.get('Cookie', '<none>>')})")
    return jsonify({"status": "ok"})

@app.route("/profiles")
def profiles():
    by_id = {}

```

```

for e in EVENTS:
    by_id.setdefault(e["analytics_id"], []).append(e)

print("\n[analytics] RECONSTRUCTED PROFILES (grouped by analytics_id)")
for aid, evts in by_id.items():
    publishers_seen = sorted(set(e["publisher"] for e in evts))
    print(f"\n    analytics_id = {aid}")
    print(f"        publishers seen under this id: {publishers_seen}")
    for e in evts:
        print(f"            {e['timestamp']} {e['publisher']:<15} {e['page']}")

combined = any(
    len(set(e["publisher"] for e in evts)) > 1 for evts in by_id.values()
)
print(f"\n => {'ONE combined cross-site profile' if combined else 'SEPARATE per-publisher profiles'} "
      f"was reconstructed from {len(by_id)} distinct analytics_id value(s).")
return jsonify(by_id)

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=9004, debug=False)

```

```

(function () {
    // 1. Check whether an analytics cookie already exists.
    //     NOTE: document.cookie here operates in the PUBLISHER's origin,
    //     not analytics.test's origin, even though this file is served
    //     from analytics.test. The script executes in the page that
    //     included it via <script src="...">.
    function getCookie(name) {
        const match = document.cookie.match(new RegExp('(^| )' + name + '=(^[^;]+)'));
        return match ? match[2] : null;
    }

    function setCookie(name, value, days) {
        const d = new Date();
        d.setTime(d.getTime() + days * 24 * 60 * 60 * 1000);
        document.cookie = `${name}=${value}; expires=${d.toUTCString()}; path=/`;
    }

    let analyticsId = getCookie('_analytics_id');
    const isNew = !analyticsId;

    // 2. Generate a random identifier if none exists yet, scoped to THIS
    //     publisher's own domain.
    if (isNew) {
        analyticsId = Math.random().toString(36).slice(2) + Date.now().toString(36);
        // 3. Store the identifier using document.cookie (first-party cookie
        //     of the publisher's domain).
        setCookie('_analytics_id', analyticsId, 365);
    }

    const publisher =
        document.currentScript && document.currentScript.getAttribute('data-publisher')
        ? document.currentScript.getAttribute('data-publisher')
        : 'unknown-publisher';

    const payload = {
        analytics_id: analyticsId,
        is_new: isNew,
        publisher: publisher,
        page: location.pathname,
        origin: location.origin,
    };

```

```
// 4. Explicitly send the identifier to the analytics server. This is
//    the ONLY way the analytics.test server learns the id: it is never
//    sent automatically, because the cookie set above belongs to the
//    publisher's domain, not to analytics.test.
fetch('http://127.0.0.1:9004/collect', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(payload),
}).catch(() => {});
})();
```

8. Conclusion

This lab demonstrated, end to end and with real running servers, how a stateless protocol like HTTP is turned into a stateful, re-identifying system through cookies: a first-party cookie is enough to remember a returning visitor on a single site; a third-party cookie embedded via an iframe lets a single tracker follow the same browser across unrelated publishers and reconstruct a chronological, cross-site profile from nothing but repeated /track hits; a first-party analytics script shows that moving tracking code into the publisher's own origin isolates identifiers per-site by default, unless the analytics provider actively works around that isolation.

Ethical note: every server in this report ran only on 127.0.0.1 (or, if you followed the original instructions, on hosts-file aliases that still resolve to your own machine), with content created solely for this exercise. These techniques must never be pointed at real, third-party websites or used to collect data about real users without explicit authorization.